



Queen Electronic Cash System

Loid Protagonist

loidprotagonist@proton.me

Abstract Queen is an open-source electronic cash system designed to enable peer-to-peer digital transactions without relying on a centralized authority. The system implements a blockchain-based ledger using an unspent transaction output (UTXO) model, cryptographic digital signatures, transaction identifiers, Merkle roots, and CPU-based Proof-of-Work mining. Transactions are validated independently by participating nodes, including verification of signatures, ownership of referenced UTXOs, transaction amounts, and prevention of double spending. Blocks are independently verified before being added to the local blockchain, allowing nodes to determine validity from the consensus rules they implement rather than trusting the miner that produced a block. Queen is implemented in Python and includes wallet, transaction, mining, blockchain, and peer-to-peer networking components. The project is currently under active development, with development focused on maintaining a simple, transparent, and independently verifiable electronic cash system.

1. Introduction

Since the introduction of Bitcoin, peer-to-peer electronic cash has demonstrated that digital value can be transferred between participants without requiring a central authority to maintain the ledger. Queen continues this approach by implementing an open-source electronic cash system based on a blockchain, unspent transaction outputs (UTXO), cryptographic signatures, and Proof-of-Work. The system is designed so that transactions and blocks are independently validated by participating nodes according to the same consensus rules. Transaction validation includes checking referenced UTXOs, ownership through public-key verification, transaction amounts, duplicate inputs, and digital signatures, while block validation includes verification of the previous chain, Merkle root, transaction identifiers, Proof-of-Work, block rewards, and other consensus conditions. Queen is implemented in Python and includes components for wallets, transactions, blockchain management, mining, and peer-to-peer communication. The project is currently under active development, with its implementation focused on providing a transparent and independently verifiable electronic cash system.

2. System Architecture

Queen consists of several interconnected components that work together to process, validate, mine, and propagate transactions across the peer-to-peer network. The Wallet manages keys and creates digitally signed transactions. The Transaction component constructs transactions using available UTXOs and prepares them for broadcast. The Node receives transactions and blocks from peers and independently validates them according to the consensus rules implemented by the system. Valid transactions are maintained in the mempool and can later be selected by the Mining component to construct a new block and perform Proof-of-Work. After a block is produced, it is verified again before being added to the local blockchain and applied to the UTXO set. The Blockchain and Consensus components provide the rules used to determine whether transactions and blocks are valid, allowing each node to independently calculate and verify the state of the chain. This architecture separates user operations, transaction processing, mining, networking, and consensus logic while keeping the validation rules consistent across participating nodes.

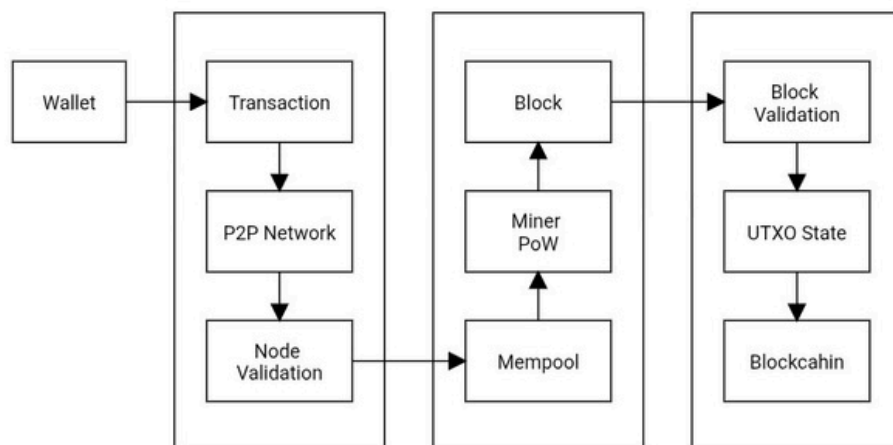


Figure 1. Queen System

Transactions are validated independently by nodes before entering the mempool, while mined blocks are independently validated before updating the blockchain and UTXO state.

3. Blockchain Design

Queen maintains a sequential blockchain in which each block contains a block height, timestamp, mining difficulty, Merkle root, transaction list, reference to the previous block, nonce, and the resulting block hash. Each transaction is identified by a transaction identifier (TXID), while transaction outputs are maintained as unspent transaction outputs (UTXOs) until they are consumed by a subsequent transaction. A transaction references an existing UTXO through its transaction identifier and output index (vout). When a valid transaction is applied, its referenced inputs are marked as spent and its outputs are added to the UTXO set. The transaction fee is determined from the difference between the total input value and the total output value.

The transaction fee is defined as:

$$F = \Sigma I - \Sigma O$$

where:

I = total value of transaction inputs

O = total value of transaction outputs

F = transaction fee

where I represents the values of all referenced UTXOs and O represents the values of all transaction outputs. A transaction is valid only when the total output value does not exceed the total input value.

The TXID is calculated by serializing the transaction without its existing TXID and applying double SHA-256:

$$\text{TXID} = \text{SHA256}(\text{SHA256}(\text{Serialized Transaction}))$$

Transactions within a block are represented by a Merkle root, allowing the node to verify that the transaction set corresponds to the Merkle root recorded in the block. The resulting block is linked to the previous block through the previous_chain field, creating a sequential chain in which modification of previously recorded data becomes detectable during independent verification.

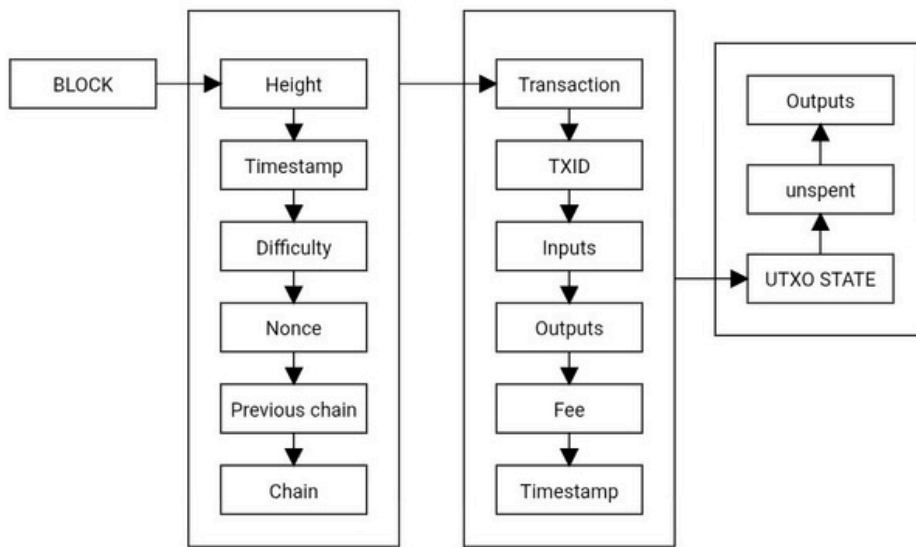


Figure 2 — Queen Blockchain Structure

Figure 2 illustrates the basic data structure of the Queen blockchain and the relationship between blocks, transactions, and the UTXO state. Each block contains the information required to identify its position in the chain, link it to the previous block, verify its Proof-of-Work, and validate its transaction set. The transaction list contains individual transactions identified by their TXIDs, with each transaction consisting of inputs and outputs. Inputs reference previously created UTXOs through a transaction ID and output index, while outputs define a new address and amount that can become an available UTXO. When a transaction is accepted, its referenced UTXOs are marked as spent and its outputs are added to the UTXO state. During block validation, the node independently checks the transactions and their referenced UTXOs before applying the resulting state changes to its local ledger. This structure allows the current spendable state of the blockchain to be derived from the transaction history rather than relying on a separate balance record.

4. Transaction Model

Queen uses a UTXO-based transaction model in which transaction inputs reference previously created unspent transaction outputs and transaction outputs create new spendable outputs. Each input identifies the previous transaction through its TXID and specifies the corresponding output index (vout). The node checks that every referenced UTXO exists and has not already been spent before accepting the transaction. The public key contained in the input is used to derive the corresponding address, which must match the address of the referenced UTXO. The transaction is then authenticated by verifying its digital signature against the transaction payload.

For a transaction with multiple inputs and outputs, the total input value is obtained from the referenced UTXOs, while the total output value is the sum of all newly created outputs:

Total Input = Sum of Referenced UTXOs

Total Output = Sum of Transaction Outputs

Transaction Fee = Total Input - Total Output

A valid transaction must satisfy:

Total Input $\geq$ Total Output

Each input may be used only once within the same transaction. If the same (txid, vout) pair appears more than once, the transaction is rejected. A referenced UTXO that does not exist or has already been marked as spent is also rejected. This prevents a transaction from consuming the same output multiple times.

After successful validation, the transaction receives its TXID and can be placed into the mempool. When the transaction is later included in a valid block, its referenced UTXOs are marked as spent and each transaction output is added to the UTXO state as a new unspent output.

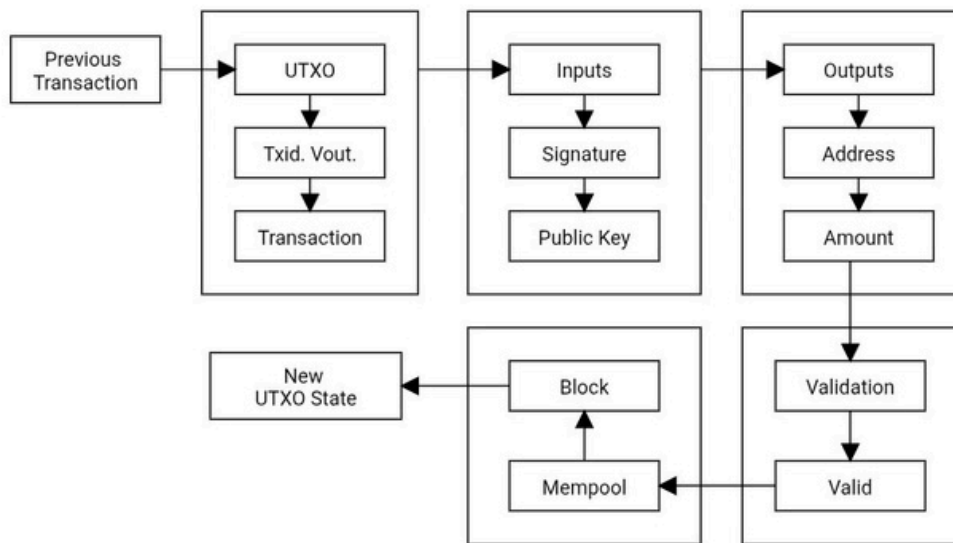
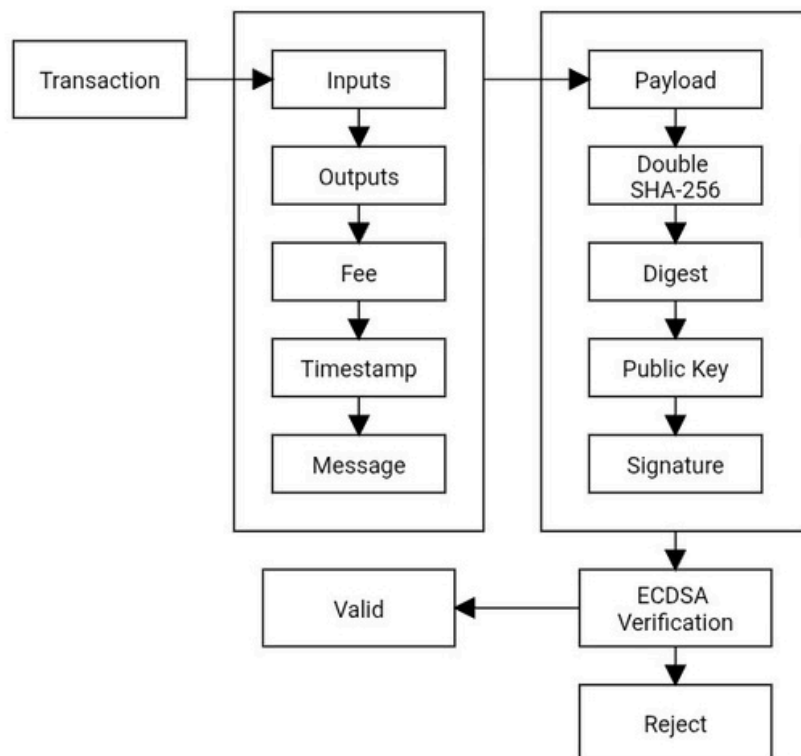


Figure 3. Transaction and UTXO Flow

Figure 3 illustrates the transaction and UTXO flow in Queen. A transaction references existing UTXOs through their transaction IDs and output indexes. The node verifies the referenced UTXOs, checks ownership through the public key, and verifies the transaction signature before accepting the transaction. The total value of the referenced inputs must be equal to or greater than the total value of the outputs, with the difference treated as the transaction fee. Duplicate inputs and already-spent or nonexistent UTXOs are rejected. Once validated, the transaction enters the mempool and can later be included in a mined block. After the block is accepted, the referenced UTXOs are marked as spent and the transaction outputs become new UTXOs available for future transactions.

5. Digital Signature and Transaction Verification

Queen uses digital signatures to authorize the spending of previously created UTXOs. Each transaction input contains a reference to a previous transaction output, a public key, and a digital signature. Before verification, Queen constructs a deterministic transaction payload containing the transaction version, input references, outputs, fee, timestamp, and message. The signature itself is not included in this payload. The serialized payload is hashed using double SHA-256 to produce the message digest used for signature verification. The public key supplied by the input is converted into an address, and the resulting address must match the address stored in the referenced UTXO. The node then verifies the signature against the calculated digest using the secp256k1 elliptic curve and DER signature decoding. If any of these checks fail, the transaction is rejected.



Verification conditions:

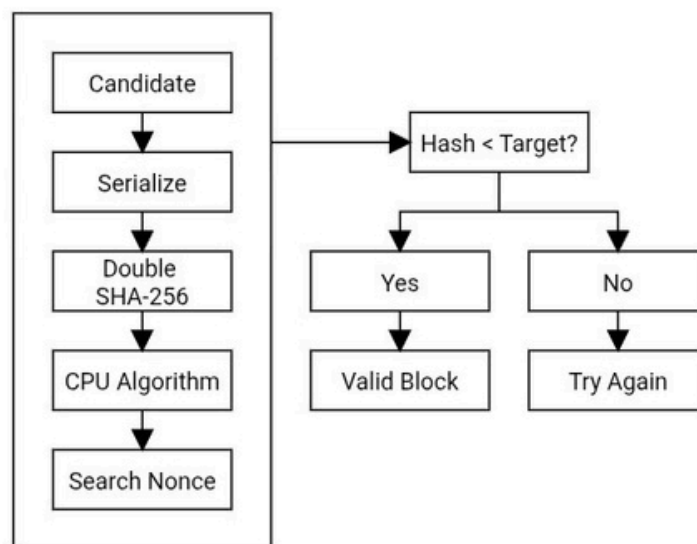
$\text{Address(Public Key)} == \text{UTXO Address}$

$\text{Signature} = \text{Verify(Public Key, Digest)}$

$\text{Valid} = \text{UTXO} + \text{Address} + \text{Signature}$

6. Proof-of-Work

Queen uses CPU-based Proof-of-Work to determine whether a candidate block satisfies the current mining difficulty. The miner first constructs a block containing the block metadata and selected transactions. The block header is serialized and processed to generate a seed, which is then supplied to the CPU mining algorithm. Multiple workers can search different nonce sequences concurrently. For each nonce, the mining algorithm produces a resulting hash that is compared with the required difficulty target. Mining succeeds when the resulting hash satisfies the target condition. The successful nonce and hash are then stored in the block. When the block is received by another node, the node independently reconstructs the required header and performs the same computation to verify the reported result before accepting the block.



Proof-of-Work condition:

Hash < Target

Figure 4. Queen Proof-of-Work Mining Process

Figure 4 illustrates the Proof-of-Work process in Queen, where miners search for a valid nonce until the resulting hash satisfies the current difficulty target. The resulting block is then independently verified by network nodes.

7. Block Validation and Consensus

A block received by a Queen node is independently validated before being added to the local blockchain. The node checks the previous block reference, block structure, Proof-of-Work, transaction validity, Merkle root, and UTXO state. Each transaction is verified independently, including its inputs, outputs, signatures, and referenced UTXOs. Blocks containing invalid transactions, invalid Proof-of-Work, inconsistent Merkle roots, or invalid chain references are rejected. This process allows nodes to reach agreement on the valid chain without relying on trust in the miner.

8. Peer-to-Peer Network

Queen uses a peer-to-peer network to distribute transactions and blocks between nodes. Valid transactions can be propagated through connected peers before being included in a block, while mined blocks are broadcast for independent validation. Nodes synchronize their blockchain state with peers and maintain their own local copy of the ledger.

9. Testing and Verification

Queen was tested by submitting both valid and invalid blockchain data to the validation system. The tests include invalid transactions, invalid signatures, incorrect transaction identifiers, invalid UTXO references, duplicate or already-spent inputs, incorrect Merkle roots, invalid Proof-of-Work, and inconsistent chain data. Additional tests were performed on competing chain conditions to verify that nodes do not simply accept a chain based on its length. These tests are used to confirm that invalid data is rejected and that valid blockchain data can be independently verified by participating nodes.

Conclusion

Queen implements a peer-to-peer electronic cash system using UTXO-based transactions, digital signatures, Proof-of-Work, and independent blockchain validation. Its design allows participating nodes to verify transactions and blocks using the same consensus rules without relying on trust in a central authority or the miner that produced a block.

References

- [1] S. Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System, 2008.

- [2] National Institute of Standards and Technology, Secure Hash Standard (SHS), FIPS PUB 180-4, 2015.

- [3] National Institute of Standards and Technology, Digital Signature Standard (DSS), FIPS PUB 186-5, 2023.

- [4] Certicom Research, SEC 2: Recommended Elliptic Curve Domain Parameters, Version 2.0, 2010.

- [5] A. M. Antonopoulos, Mastering Bitcoin: Programming the Open Blockchain, 2nd ed., O'Reilly Media, 2017.

- [6] Python Software Foundation, Python 3 Documentation.

- [7] Loid Protagonist, Queen Electronic Cash System — Source Code, 2026.